```
from fastapi import FastAPI, Query
from fastapi.middleware.cors import CORSMiddleware
from datetime import datetime, timedelta
from typing import List, Dict, Optional
import httpx
import asyncio

app = FastAPI(title="המלצות היומיות מאת רון")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

FOOTBALL_API_KEY = "f6eb9f6501d64379a504e613f2261fec"
FOOTBALL_BASE    = "https://api.football-data.org/v4"
HEADERS          = {"X-Auth-Token": FOOTBALL_API_KEY}

COMPETITIONS = {
    "PL":  "פרמייר ליג – אנגליה 🏴",
    "PD":  "לה ליגה – ספרד 🇪🇸",
    "SA":  "סריה א – איטליה 🇮🇹",
    "BL1": "בונדסליגה – גרמניה 🇩🇪",
    "FL1": "ליג 1 – צרפת 🇫🇷",
    "DED": "ארדיביזי – הולנד 🇳🇱",
    "PPL": "פרימיירה ליגה – פורטוגל 🇵🇹",
    "CL":  "ליגת האלופות UEFA 🌍",
    "EL":  "ליגה אירופית UEFA 🌍",
    "BSA": "ברזילאו – ברזיל 🇧🇷",
    "CLI": "קופה ליברטדורס 🌎",
}

# ─── ניתוח סטטיסטי ───

def calc_form_score(matches: list, team_id: int) -> dict:
    points = goals_scored = goals_conceded = 0
    results = []
    for m in matches[-5:]:
        home_id = m["homeTeam"]["id"]
        ft = (m.get("score") or {}).get("fullTime") or {}
        gh = ft.get("home") or 0
        ga = ft.get("away") or 0
```

```
            if team_id == home_id:
                goals_scored   += gh; goals_conceded += ga
                if gh > ga:    points += 3; results.append("W")
                elif gh == ga: points += 1; results.append("D")
                else:          results.append("L")
            else:
                goals_scored   += ga; goals_conceded += gh
                if ga > gh:    points += 3; results.append("W")
                elif ga == gh: points += 1; results.append("D")
                else:          results.append("L")
        n = len(matches[-5:]) or 1
        return {
            "points":       points,
            "form_string":  " ".join(results) if results else "אין נתונים",
            "avg_scored":   round(goals_scored   / n, 2),
            "avg_conceded": round(goals_conceded  / n, 2),
            "form_pct":     round(points / (n * 3) * 100),
        }

    def calc_h2h(h2h_matches: list, home_id: int, away_id: int) -> dict:
        hw = aw = d = 0
        for m in h2h_matches[-10:]:
            ft = (m.get("score") or {}).get("fullTime") or {}
            gh = ft.get("home") or 0
            ga = ft.get("away") or 0
            hid = m["homeTeam"]["id"]
            if hid == home_id:
                if gh > ga:   hw += 1
                elif gh < ga: aw += 1
                else:         d  += 1
            else:
                if ga > gh:   hw += 1
                elif ga < gh: aw += 1
                else:         d  += 1
        total = hw + aw + d or 1
        return {
            "home_wins": hw, "away_wins": aw, "draws": d, "total": total,
            "home_win_pct": round(hw / total * 100),
            "away_win_pct": round(aw / total * 100),
            "draw_pct":     round(d  / total * 100),
        }

    def interpret(score: int) -> str:
        if score >= 9: return "כמעט בטוח ✅"
        if score >= 7: return "מומלץ מאוד ⭐"
        if score >= 5: return "אפשרי ⚠️"
        return "לא מומלץ ❌"
```

```
def generate_analysis(hf: dict, af: dict, h2h: dict, match: dict) -> list:
    hn = match["homeTeam"]["name"]
    an = match["awayTeam"]["name"]
    recs = []

    # ── 1. ניצחון / תיקו / הפסד ──
    hs = hf["form_pct"] * 0.5 + h2h["home_win_pct"] * 0.3 + (hf["avg_scored"] - h
    as_ = af["form_pct"] * 0.5 + h2h["away_win_pct"] * 0.3 + (af["avg_scored"] -
    if hs > as_ + 15:
        pred  = f"ניצחון {hn}"; sc = min(10, max(4, int(hs / 12)))
        reason = f"פורמה ביתית {hf['form_pct']}% | עימותים {h2h['home_win_pct']}%
    elif as_ > hs + 15:
        pred  = f"ניצחון {an}"; sc = min(10, max(4, int(as_ / 12)))
        reason = f"פורמה אורחת {af['form_pct']}% | עימותים {h2h['away_win_pct']}%
    else:
        pred  = "תיקו"; sc = min(8, max(4, int(h2h["draw_pct"] / 10) + 3))
        reason = f"תיקו | פורמה דומה לשתי הקבוצות | עימותים {h2h['draw_pct']}%"
    recs.append({"type": "ניצחון/תיקו/הפסד", "prediction": pred, "score": sc,
                 "interpretation": interpret(sc), "reasoning": reason,
                 "odds": round(100 / max(sc * 10, 20), 2)})

    # ── 2. מעל/מתחת 2.5 גולים ──
    avg = (hf["avg_scored"] + hf["avg_conceded"] + af["avg_scored"] + af["avg_con
    sc2 = min(10, max(4, int(avg * 2.2)))
    pred2 = "מעל 2.5 גולים" if avg > 1.3 else "מתחת 2.5 גולים"
    recs.append({"type": "מעל/מתחת 2.5 גולים", "prediction": pred2, "score": sc2,
                 "interpretation": interpret(sc2),
                 "reasoning": f"ממוצע {avg:.1f} גולים למשחק לפי 5 משחקים אחרונים"
                 "odds": round(100 / max(sc2 * 10, 20), 2)})

    # ── 3. שתי הקבוצות מבקיעות ──
    btts = min(hf["avg_scored"], 1.5) * min(af["avg_scored"], 1.5)
    sc3  = min(10, max(4, int(btts * 5)))
    pred3 = "כן — שתיהן מבקיעות" if btts > 0.8 else "לא — לפחות אחת לא מבקיעה"
    recs.append({"type": "שתי הקבוצות מבקיעות (BTTS)", "prediction": pred3, "scor
                 "interpretation": interpret(sc3),
                 "reasoning": f"{hn}: {hf['avg_scored']} ממוצע גולים | {an}: {af[
                 "odds": round(100 / max(sc3 * 10, 20), 2)})

    return sorted(recs, key=lambda x: x["score"], reverse=True)

# ── קריאות API ──

async def fetch(client: httpx.AsyncClient, path: str) -> dict:
    try:
        r = await client.get(f"{FOOTBALL_BASE}{path}", headers=HEADERS, timeout=1
```

```
        return r.json() if r.status_code == 200 else {}
    except Exception:
        return {}

async def get_matches(client, comp_id: str, date_str: str) -> list:
    data = await fetch(client, f"/competitions/{comp_id}/matches?dateFrom={date_s
    return data.get("matches", [])

async def enrich(client: httpx.AsyncClient, match: dict, date_str: str) -> Option
    home_id  = match["homeTeam"]["id"]
    away_id  = match["awayTeam"]["id"]
    match_id = match["id"]

    hd, ad, h2hd = await asyncio.gather(
        fetch(client, f"/teams/{home_id}/matches?status=FINISHED&limit=5"),
        fetch(client, f"/teams/{away_id}/matches?status=FINISHED&limit=5"),
        fetch(client, f"/matches/{match_id}/head2head?limit=10"),
    )

    hm = hd.get("matches", [])
    am = ad.get("matches", [])
    h2hm = h2hd.get("matches", [])

    hf  = calc_form_score(hm,   home_id)
    af  = calc_form_score(am,   away_id)
    h2h = calc_h2h(h2hm, home_id, away_id)
    recs = generate_analysis(hf, af, h2h, match)

    utc  = match.get("utcDate", "")
    kick = utc[11:16] if len(utc) >= 16 else "--:--"

    return {
        "id":           match_id,
        "competition":  match.get("competition", {}).get("name", ""),
        "comp_code":    match.get("competition", {}).get("code", ""),
        "home_team":    match["homeTeam"]["name"],
        "away_team":    match["awayTeam"]["name"],
        "game":         f"{match['homeTeam']['name']} נגד {match['awayTeam']['nam
        "kickoff":      kick,
        "date":         date_str,
        "home_form":    hf,
        "away_form":    af,
        "h2h":          h2h,
        "recommendations": recs,
    }

# ── Endpoints ──
```

```
@app.get("/games/{day_offset}")
async def get_games(
    day_offset: int = 0,
    comp: Optional[str] = Query(None),
    min_score: Optional[int] = Query(None),
):
    if not 0 <= day_offset <= 3:
        return {"error": "ניתן לבחור רק 0-3 ימים קדימה"}

    date_str = (datetime.utcnow() + timedelta(days=day_offset)).strftime("%Y-%m-%
    comp_ids = [comp] if comp and comp in COMPETITIONS else list(COMPETITIONS.key

    async with httpx.AsyncClient() as client:
        lists = await asyncio.gather(*[get_matches(client, c, date_str) for c in
        all_matches = [m for lst in lists for m in lst]
        if not all_matches:
            return []
        enriched = await asyncio.gather(*[enrich(client, m, date_str) for m in al

    result = [g for g in enriched if g]
    if min_score:
        result = [g for g in result if any(r["score"] >= min_score for r in g["re
    return sorted(result, key=lambda x: x["kickoff"])

@app.get("/competitions")
def get_competitions():
    return COMPETITIONS

@app.get("/stats/{day_offset}")
async def get_stats(day_offset: int = 0):
    date_str = (datetime.utcnow() + timedelta(days=day_offset)).strftime("%Y-%m-%
    async with httpx.AsyncClient() as client:
        lists = await asyncio.gather(*[get_matches(client, c, date_str) for c in
    total = sum(len(l) for l in lists)
    return {"total_games": total, "date": date_str, "competitions": len(COMPETITI
```